INSTITUTO FEDERAL
Alagoas
Campus
Arapiraca

INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA DE ALAGOAS
CAMPUS ARAPIRACA

CURSO SUPERIOR EM SISTEMAS DE INFORMAÇÃO

Administração e Projeto de Banco de Dados

Jefferson Santos de Farias
José Arthur da Silva Teles

ARAPIRACA - AL
Dezembro / 2025

Jefferson Santos de Farias
José Arthur da Silva Teles

Relatório de Administração e Projeto de Banco de Dados

Relatório baseado na Atividade Prática Final, requisitado para obtenção de nota na disciplina de Administração e Projeto de Banco de Dados do curso de Sistemas de Informação.

ARAPIRACA - AL
Dezembro / 2025

Introdução

Este relatório apresenta o resultado da atividade prática final da disciplina de Administração e Projeto de Banco de Dados, aplicada ao cenário da empresa Olist. O objetivo principal deste trabalho foi simular a rotina de um administrador de dados, transformando informações brutas de vendas e logística em um banco de dados organizado, seguro e pronto para uso corporativo. O sistema gerenciador de banco de dados escolhido foi o MySQL devido à familiaridade com a sintaxe proveniente das aulas anteriores da disciplina.

O desenvolvimento do projeto seguiu um fluxo lógico dividido em três grandes etapas. Inicialmente, foi realizada a estruturação e a importação dos dados, garantindo que as informações de clientes, produtos e pedidos estivessem corretamente conectadas e sem erros. Em seguida, o foco passou a ser a segurança e a inteligência de negócios: criamos um usuário para que a equipe de BI (Business Intelligence) pudesse consultar os dados sem riscos de alterações acidentais e desenvolver relatórios estratégicos sobre o desempenho de vendas.

Por fim, o documento detalha as melhorias de performance aplicadas para tornar o sistema mais rápido, bem como as estratégias de auditoria e backup definidas para garantir que as informações da empresa estejam sempre protegidas e disponíveis. Os primeiros módulos referentes a criação de tabelas e importação dos dados CSV e o módulo referente a consultas estratégicas estarão sendo enviados como arquivos .sql na atividade no classroom junto com o relatório.

1. Restrições de Integridade

1.1 Restrições de chaves estrangeiras:

```
ALTER TABLE pedidos
ADD CONSTRAINT fk_pedidos_cliente
FOREIGN KEY (id_cliente) REFERENCES CLIENTES(id_cliente)
ON DELETE RESTRICT
ON UPDATE CASCADE;
```

Ação	Cláusula SQL	Justificativa
Exclusão	`ON DELETE RESTRICT;`	Um cliente não deve ser deletado se tiver pedidos no histórico.
Atualização	`ON UPDATE CASCADE;`	Se o `id_cliente` for alterado, o sistema deve refletir essa mudança automaticamente.

```
ALTER TABLE ITENS_PEDIDO
ADD CONSTRAINT fk_itens_pedido
FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

Ação	Cláusula SQL	Justificativa
Exclusão	`ON DELETE CASCADE;`	Se um pedido for excluído, os itens devem ser deletados automaticamente.
Atualização	`ON UPDATE CASCADE;`	Se o `id_pedido` for alterado, o mesmo deve ser atualizado em cascata em todos os registros de `ITENS_PEDIDO`.

```
ALTER TABLE ITENS_PEDIDO
ADD CONSTRAINT fk_itens_produto
FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)
ON DELETE RESTRICT
ON UPDATE CASCADE;
```

Ação	Cláusula SQL	Justificativa
Exclusão	`ON DELETE RESTRICT;`	Um produto não deve ser excluído da tabela `PRODUTOS` se já existir um registro de venda associado a ele.
Atualização	`ON UPDATE CASCADE;`	Se o `id_produto` for alterado, o mesmo deve ser atualizado automaticamente em todos os registros de `ITENS_PEDIDO`.

```sql
ALTER TABLE ITENS_PEDIDO
ADD CONSTRAINT fk_itens_vendedor
FOREIGN KEY (id_vendedor) REFERENCES vendedores(id_vendedor)
ON DELETE RESTRICT
ON UPDATE CASCADE;
```

Ação	Cláusula SQL	Justificativa
Exclusão	`ON DELETE RESTRICT;`	Um vendedor não deve ser excluído se ainda existir um registro de venda associado a ele.
Atualização	`ON UPDATE CASCADE;`	Se o `id_vendedor` for alterado, o mesmo deve ser atualizado em cascata em todos os registros de `ITENS_PEDIDO`.

```sql
ALTER TABLE pagamentos
ADD CONSTRAINT fk_pagamentos_pedido
FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido)
ON DELETE CASCADE
ON UPDATE CASCADE;

ALTER TABLE avaliacoes
ADD CONSTRAINT fk_avaliacoes_pedido
FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

Tabela filha	Ação	Cláusula SQL	Justificativa
`pagamentos`	Exclusão	`ON DELETE CASCADE;`	Se um pedido for excluído, o pagamento referente a ele deve ser excluído junto.
`avaliacoes`	Exclusão	`ON DELETE CASCADE;`	Se um pedido for excluído, a avaliação perde o contexto e deve ser excluída em cascata.
`pagamentos` `avaliacoes`	Atualização	`ON UPDATE CASCADE;`	Se o `id_pedido` for alterado, o mesmo deve ser atualizado em cascata em todos os registros de `PAGAMENTOS` e `AVALIACOES`.

```sql
ALTER TABLE produtos
ADD CONSTRAINT fk_produtos_categoria
FOREIGN KEY (categoria_produto) REFERENCES traducao_categorias(categoria_produto)
ON DELETE SET NULL
ON UPDATE CASCADE;
```

Ação	Cláusula SQL	Justificativa
Exclusão	`ON DELETE SET NULL;`	Se uma categoria for removida, os produtos que a referenciam não devem ser deletados. Em vez disso, o campo `categoria_produto` deve ser definido como `NULL`.
Atualização	`ON UPDATE CASCADE;`	Se o `categoria_produto` for alterado, o mesmo deve ser atualizado em cascata em todos os registros de `PRODUTOS`.

1.2 Índices:

```sql
CREATE INDEX idx_pedidos_cliente ON pedidos(id_cliente);
```

Este índice otimiza a velocidade de busca e recuperação de todos os pedidos realizados por um cliente específico. Ele é essencial para consultas que filtram, agrupam ou ordenam dados pelo `id_cliente`.

```sql
CREATE INDEX idx_produtos_categoria ON produtos(categoria_produto);
```

Este índice melhora a performance de consultas que visam filtrar ou agrupar produtos com base em sua categoria, suportando a navegação do catálogo e a análise de vendas por segmento.

1.3 Restrições de Domínio:

```sql
ALTER TABLE ITENS_PEDIDO
ADD CONSTRAINT check_preco_positivo CHECK (preco >= 0),
ADD CONSTRAINT check_frete_positivo CHECK (valor_frete >= 0);

ALTER TABLE pagamentos
ADD CONSTRAINT check_pagamento_positivo CHECK (valor_pagamento >= 0);
```

Garantem que o preço unitário, valor do frete e o pagamento sejam armazenados como valores monetários não-negativos.

```sql
ALTER TABLE produtos
ADD CONSTRAINT check_peso_valido CHECK (peso_produto_g > 0),
ADD CONSTRAINT check_dimensoes_validas CHECK (comprimento_produto_cm > 0 AND
altura_produto_cm > 0 AND largura_produto_cm > 0);
```

Garante que o peso e as dimensões de um produto sejam sempre positivos. Essencial para o cálculo logístico e de volume.

```sql
ALTER TABLE avaliacoes
ADD CONSTRAINT check_pontuacao_range CHECK (pontuacao BETWEEN 1 AND 5);
```

Garante que a pontuação dada em uma avaliação esteja dentro do domínio aceitável (de 1 a 5), padronizando a escala de rating do sistema.

2. Criar um usuário para o pessoal de BI

```sql
CREATE USER 'userBI'@'%' IDENTIFIED BY 's3nh4';
GRANT SELECT ON olist.* TO 'userBI'@'%';
FLUSH PRIVILEGES;
```

Para atender às necessidades de análise de dados e Business Intelligence (BI), foi criado um usuário específico com permissões restritas. O usuário userBI foi estabelecido para ser acessado de qualquer host ('%') e tem a função única de realizar consultas para relatórios e análises. A permissão concedida é estritamente `SELECT (GRANT SELECT ON olist.*)`, o que significa que o usuário só pode ler os dados de todas as tabelas dentro do banco olist, sendo incapaz de modificar, inserir ou excluir qualquer informação do banco. Esta medida de segurança implementa o princípio do privilégio mínimo, garantindo que as ferramentas e sistemas de BI possam extrair os dados necessários sem representar um risco à integridade transacional ou operacional da base de dados. Por fim, o comando `FLUSH PRIVILEGES` é executado para garantir que as novas permissões entrem em vigor imediatamente.

3. Otimização de Consultas

Consulta: 5.6 Pedidos com mais atrasos por período (Top 10)

O banco de dados precisava ler todas as 99 mil linhas da tabela para verificar quais pedidos estavam 'entregues' e se eram de '2017'. Ao criarmos o índice, organizamos a tabela internamente como se fosse uma lista telefônica: agrupada primeiro pelo sttus em seguida, ordenada pela data. Com isso, o banco de dados parou de procurar em toda a tabela e passou a ir direto ao bloco de dados que atende aos critérios, reduzindo drasticamente o tempo de busca.

Consulta Original:

```
-- Medindo o tempo sem índices otimizados
EXPLAIN ANALYZE
SELECT p.id_pedido, p.data_estimada_entrega, p.data_entrega_cliente,
     DATEDIFF(p.data_entrega_cliente, p.data_estimada_entrega) AS
dias_de_atraso
FROM pedidos p
WHERE p.status_pedido = 'delivered'
AND p.data_entrega_cliente > p.data_estimada_entrega
AND p.data_estimada_entrega BETWEEN '2017-01-01' AND '2017-12-31'
ORDER BY dias_de_atraso DESC
LIMIT 10;
```

O banco vai ler todos os 99 mil pedidos, verificar um por um se é 'delivered' e depois checaer as datas.

Tempo: 300ms Podendo ser maior em máquinas comuns.

Otimização:

```
-- Criando o índice que cobre as duas colunas usadas no WHERE
CREATE INDEX idx_pedidos_status_estimativa
ON pedidos(status_pedido, data_estimada_entrega);
```

Agora rodamos o `EXPLAIN ANALYZE` novamente.

Tempo: 10ms aproximados.

Consulta: 5.7 Clientes com Maior Valor em Compras (Top 10)

Nesta consulta, o banco de dados precisava realizar um esforço enorme para cruzar informações de três tabelas diferentes (Clientes, Pedidos e Itens), verificando linha por linha para encontrar as correspondências.

Consulta Original:

```
EXPLAIN ANALYZE
SELECT c.id_unico_cliente,
COUNT(DISTINCT p.id_pedido) AS total_pedidos,
SUM(i.preco) AS valor_total_gasto
FROM pedidos p
INNER JOIN CLIENTES c ON p.id_cliente = c.id_cliente
INNER JOIN ITENS_PEDIDO i ON p.id_pedido = i.id_pedido
WHERE p.data_compra BETWEEN '2017-01-01 00:00:00' AND '2017-12-31 23:59:59'
GROUP BY c.id_unico_cliente
ORDER BY valor_total_gasto DESC
LIMIT 10;
```

Tempo: 1000ms

Otimização:

```
CREATE INDEX idx_pedidos_data_compra ON pedidos(data_compra);
CREATE INDEX idx_itens_pedido_id ON ITENS_PEDIDO(id_pedido, preco);
```

Tempo: 50ms

- Criamos um índice na Data, o que funcionou como um filtro prévio: o banco passou a ignorar imediatamente todos os pedidos fora do ano de 2017, reduzindo o volume de trabalho.
- Indexamos as colunas de ligação: Chaves Estrangeiras. Isso permitiu que o banco encontrasse o preço e o cliente associado a cada pedido instantaneamente, sem precisar fazer buscas demoradas nas outras tabelas.

Resultado: O tempo de resposta caiu porque eliminamos o trabalho desnecessário de ler dados antigos e agilizamos o cruzamento das informações. O banco filtrou imediatamente o ano de 2017 e encontrou os preços dos itens instantaneamente através dos índices, sem precisar percorrer as tabelas inteiras. Isso eliminou o gargalo de processamento e reduziu o tempo de entrega da consulta em mais de 90%.

Existem outras formas de melhorar a performance das consultas? Se sim, o que você poderia sugerir como mudança?

Sim. Com o passar dos anos, a tabela pedidos tende a crescer sempre. Consultas do dia a dia como "Vendas do mês atual"começam a ficar lentas porque o banco precisa lidar com milhões de registros de anos anteriores que raramente são consultados. Sendo uma possível solução, implementar uma rotina automática de arquivamento. Dados de pedidos concluídos há mais de 2 ou 5 anos serão movidos da tabela principal “pedidos “ para uma tabela de histórico, assim aliviando o banco.

3.1 Otimização com Materialized Views

Consulta escolhida:

```
-- 3. Média de avaliações por loja:
EXPLAIN ANALYZE
SELECT v.id_vendedor,
ROUND(AVG(a.pontuacao), 2) AS media_avaliacao
FROM vendedores v
INNER JOIN ITENS_PEDIDO i ON v.id_vendedor = i.id_vendedor
INNER JOIN avaliacoes a ON i.id_pedido = a.id_pedido
GROUP BY v.id_vendedor, v.cidade_vendedor
ORDER BY media_avaliacao DESC;
```

A consulta original custou 2,04 segundos. O MySQL não tem a função “materialized view” nativa, então criaremos uma tabela que terá os resultados pré-processados.

Tabela nova:

```
CREATE TABLE mv_relatorio_vendedores (
   id_vendedor VARCHAR(50) PRIMARY KEY,
   cidade_vendedor VARCHAR(100),
   qtd_avaliacoes INT,
   media_avaliacao DECIMAL(3, 2),
   data_atualizacao DATETIME
);
```

Índice para otimização da tabela:

```
CREATE INDEX idx_mv_media ON mv_relatorio_vendedores(media_avaliacao);
```

Consulta nova (12,25 ms):

```
EXPLAIN ANALYZE
SELECT id_vendedor, cidade_vendedor, qtd_avaliacoes, media_avaliacao
FROM mv_relatorio_vendedores
ORDER BY media_avaliacao DESC
LIMIT 100;
```

Novo INSERT:

```
INSERT INTO CLIENTES (id_cliente, id_unico_cliente, cep_cliente,
cidade_cliente, estado_cliente)
VALUES ('cli_teste_001', 'uniq_001', '00000', 'Teste City', 'TS');

INSERT INTO vendedores (id_vendedor, cep_vendedor, cidade_vendedor,
estado_vendedor)
VALUES ('vend_teste_001', '00000', 'Teste City', 'TS');

INSERT INTO produtos (id_produto, categoria_produto, peso_produto_g,
comprimento_produto_cm, altura_produto_cm, largura_produto_cm)
VALUES ('prod_teste_001', 'outros', 500, 10, 10, 10);

INSERT INTO pedidos (id_pedido, id_cliente, status_pedido, data_compra)
VALUES ('ped_teste_001', 'cli_teste_001', 'delivered', NOW());

INSERT INTO ITENS_PEDIDO (id_pedido, id_item_pedido, id_produto, id_vendedor,
preco, valor_frete)
VALUES ('ped_teste_001', 1, 'prod_teste_001', 'vend_teste_001', 100.00,
20.00);

INSERT INTO avaliacoes (id_avaliacao, id_pedido, pontuacao,
titulo_comentario, data_criacao)
VALUES ('av_teste_001', 'ped_teste_001', 5, 'Otimo vendedor', NOW());
```

Select na view:

```
SELECT * FROM mv_relatorio_vendedores
WHERE id_vendedor = 'vend_teste_001';
```

A nova avaliação não apareceu, pois a nova tabela criada (`mv_relatorio_vendedores`) é uma tabela física e não uma view.Ou seja, seria como se a tabela fosse uma "fotografia" e não se atualiza sozinha quando os dados da tabela original mudam.

Para resolver seria necessário forçar um REFRESH na view materializada, limpando a tabela antiga usando "TRUNCATE TABLE" e recarregando com os dados atualizados.

4. Auditoria no BD

Para garantir a auditoria dos dados pode-se usar as tabelas de logs alimentadas por TRIGGERS. A ideia é criar uma tabela centralizada que receba um registro sempre que um INSERT, DELETE e UPDATE ocorrerem.

Justificativas:

- Mesmo que o dado seja alterado manualmente, as TRIGGERS sempre irão gravar no log;
- É registrado o usuário do banco, o timestamp exato e o estado dos dados antes e depois da alteração;
- Se o TRIGGER falhar a transação é interrompida;

Aplicação:

```sql
CREATE TABLE logs_auditoria (
   id_log INT AUTO_INCREMENT PRIMARY KEY,
   nome_tabela VARCHAR(50),
   acao VARCHAR(10),
   chave_primaria VARCHAR(50),
   usuario_banco VARCHAR(100),
   data_hora DATETIME DEFAULT NOW(),
   dados_antigos JSON,
   dados_novos JSON
);

DELIMITER $$
CREATE TRIGGER trg_auditoria_produtos_update
AFTER UPDATE ON produtos FOR EACH ROW
BEGIN
   INSERT INTO logs_auditoria (
       nome_tabela,
       acao,
       chave_primaria,
       usuario_banco,
       dados_antigos,
       dados_novos
   )
   VALUES (
       'produtos',
       'UPDATE',
       OLD.id_produto,
       CURRENT_USER(),
       JSON_OBJECT(
           'categoria', OLD.categoria_produto,
           'peso', OLD.peso_produto_g
       ),
       JSON_OBJECT(
           'categoria', NEW.categoria_produto,
           'peso', NEW.peso_produto_g
       )
   );
END$$
DELIMITER ;
```

Exemplo prático:

```sql
UPDATE produtos
SET peso_produto_g = 2500,
categoria_produto = 'eletronicos_premium'
WHERE id_produto = 'prod_teste_001';

SELECT * FROM logs_auditoria ORDER BY data_hora DESC;
```

id_log	1
nome_tabela	produtos
acao	UPDATE
chave_primaria	prod_teste_001
usuario_banco	root@localhost
data_hora	2025-11-27 17:09:43
dados_antigos	{"peso": 500, "categoria": "outros"}
dados_novos	{"peso": 2500, "categoria": "eletronicos_premium"}

5. Estratégia de backup

Qual estratégia seria adotada?
Faremos um backup completo diário. Como funciona: Todo dia, em um horário de pouco uso exemplo: 03:00 da manhã, o sistema faz uma cópia exata de todo o banco de dados.
Por que essa escolha? É a mais simples de implementar e de recuperar. Se o servidor quebrar na terça-feira, basta pegar o arquivo de segunda-feira à noite e restaurar. Não precisa juntar pedaços de arquivos diferentes.

Onde seria armazenado?
Em um serviço de nuvem exemplo: Google Drive, OneDrive ou AWS. O arquivo é gerado no servidor e imediatamente enviado para uma pasta na nuvem. Isso garante que,se o computador/servidor queimar ou for roubado, os dados estão salvos em outro lugar físico.

Como validaria?
Fazendo um teste de restauração mensal. Não adianta só ter o arquivo, tem que ver se ele abre. O teste consiste em uma vez por mês, o administrador pega o último backup, restaura em um computador de teste e faz um SELECT COUNT(*) na tabela de Pedidos. Se os dados aparecerem, o backup está validado.

REFERÊNCIAS

1. W3SCHOOLS. *SQL Tutorial.* Refsnes Data, 2025. Disponível em: https://www.w3schools.com/sql/. Acesso em: 01 nov. 2025.

2. MYSQL. *MySQL 8.4 Reference Manual.* Oracle Corporation, 2025. Disponível em: https://dev.mysql.com/doc/refman/8.4/en/. Acesso em: 11 nov. 2025.

3. LIMA, Marcelo. *Índices no MySQL: Otimização de Consultas.* Linha de Código, 2012. Disponível em: http://www.linhadecodigo.com.br/artigo/3620/indices-mysql-otimizacao-de-consultas.aspx. Acesso em: 11 nov. 2025.

4. FIVE.CO. *Materialized Views in MySQL.* Blog Five.co, [s.d.]. Disponível em: https://five.co/blog/materialized-views-in-mysql/. Acesso em: 15 nov. 2025.

5. Monte Carlo Data. *Guide to Domain Integrity in Databases.* MonteCarloData Blog, [s.d.]. Disponível em: https://www-montecarlodata-com.translate.goog/blog-guide-to-domain-integrity-in-databases/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc. Acesso em: 15 nov. 2025.

6. HOSTINGER. *Como criar usuário MySQL e conceder privilégios.* Hostinger Tutoriais, 2025. Disponível em: https://www.hostinger.com/br/tutoriais/como-criar-usuario-mysql-e-conceder-privilegios. Acesso em: 20 nov. 2025.